



**AIR FORCE SCHOOL, 9BRD
CHANDAN NAGAR, PUNE**

PROJECT REPORT ON

Threadsorama

SUBJECT: COMPUTER SCIENCE

2024-2025

NAME: Mannat Bhatt

CLASS: 12TH

STREAM: Science

Roll NO:

Seat no:

CERTIFICATE



This is to certify that, Miss. **Mannat Bhatt** of Class XII Science of Air Force School, 9BRD Chandannagar, Pune has completed her project file under my supervision. She has taken proper care and shown utmost sincerity in completion of this project.

I certify this project is up to my expectation as per the guidelines issued by CBSE.

**Subject Teacher
Sign**

**Principal's
Sign**

**External Examiner
Sign**

ACKNOWLEDGEMENT

I want to express my sincere appreciation to Mrs. Shivangini Shrivastava, my Computer Science teacher, for her invaluable suggestions and guidance throughout this project. She has been a constant source of inspiration, helping me grasp crucial project details and providing the necessary motivation. The success of my project is entirely attributed to her unwavering support and mentorship.

Mannat Bhatt

Class XII Science

INDEX

Sr. No	Content	Page N0
01	Introduction Of the Project	5
02	Functions Used	11
03	Working Description	13
04	Code	15-61
05	Output	62-66
06	Conclusion	67-69
07	Bibliography	70

INTRODUCTION

A) OBJECTIVE:

In a world where fashion trends change faster than the seasons, our mission is to bridge the gap between style and sustainability. We understand the importance of repurposing fashion to minimize our carbon footprint and contribute to a more circular economy. Textile waste exacerbates landfill overflow, consuming valuable space and emitting methane gas during the decomposition of natural fibres. That why we introduce THREADSORAMA. Threadsorama specializes in upcycling and repurposing old clothes. They collect old clothes, then their skilled designers and tailors work their magic to turn them into new, fashionable pieces. Here at Threadsorama clothing store we sell clothes for all ages, sizes, and occasions. We also sell accessories such as bags, wallets, shoes, and jewellery.

Our project focuses on designing and implementing a secure, scalable database and web application for our sustainable fashion company specializing in upcycling and repurposing used clothing.

The project focuses on 5 major sections:

- i. Order Data management
- ii. Customer Data management

- iii. Product Data management
- iv. Customer Pickup Data Management
- v. Stock Management

B) SOFTWARE REQUIREMENTS:

1. Provide a GUI to the customer to buy products and book pickup for recyclable clothes
2. Create the following tables in the database shoppingportal using tables.py file
 - Customer details
 - Product Category
 - Product
 - Stock
 - Order Details
 - Delivery Details
 - Pickup Details
3. A Python script to run the GUI window.

C) MODULE DESCRIPTION:

DATETIME MODULE:

Python's datetime module offers a comprehensive toolkit packed with features designed to streamline your workflow:

- **Object-Oriented Approach:** Dates, times, and time intervals are represented as objects, enabling natural manipulation and formatting.
- **Flexible Formatting:** Customize date and time output using strftime directives to perfectly match your project's needs.
- **Timezone Awareness:** Effortlessly handle time zones with timezone objects, ensuring accuracy across global applications.
- **Time Arithmetic Made Easy:** Perform calculations like adding or subtracting days, weeks, months, or even years from dates with ease.

SUBPROCESSES MODULE:

The subprocess module empowers you to create and manage new processes. Think of it as a way to execute other programs or commands directly from your Python code, like opening applications, sending them data, and receiving results. This makes it incredibly useful for:

- Automating tasks: Programmatically interact with your computer, eliminating repetitive manual commands.
- Integrating external programs: Seamlessly incorporate other tools or scripts into your Python projects.
- Executing shell commands: Run terminal commands directly from your Python code, enhancing flexibility.

The subprocess module offers two main approaches:

- run function: A convenient high-level wrapper for common use cases, launching a process, capturing its output and error, and waiting for it to finish.
- Popen class: Provides finer-grained control over process creation, allowing for complex interactions like redirecting input and output streams, managing signals, and more.

TKINTER MODULE:

KINTER, built-in with Python, offers a powerful yet straightforward approach to crafting graphical user interfaces (GUIs) for your applications:

- Effortless Integration: No external libraries needed, it comes bundled with Python itself.

- **Cross-Platform Prowess:** Your applications seamlessly adapt to Windows, macOS, and Linux systems.
- **Widget Wonderland:** From buttons and text boxes to menus and sliders, choose from a diverse set of UI elements.
- **Layout Mastery:** Arrange your widgets with precision using geometry managers like pack, grid, and place.
- **Event-Driven Magic:** Respond to user interactions like clicks, key presses, and more, making your GUI dynamic.
- **Theming Options:** Enhance the visual appeal of your applications with various theming options.

While some consider its appearance "classic," TKINTER's ease of use and extensive documentation make it an excellent choice for beginners and experienced developers alike. It's a solid foundation for building user-friendly and interactive Python applications.

MYSQLCONNECTOR:

MySQL Connector/Python acts as a bridge between your Python applications and MySQL databases, enabling seamless communication and data exchange:

- **Connection Establishment:** Establishes secure connections to MySQL servers using credentials and configuration settings.
- **SQL Execution:** Executes SQL statements (SELECT, INSERT, UPDATE, DELETE, etc.) to retrieve, manipulate, and manage data within the database.
- **Result Fetching:** Retrieves and processes results from executed queries, allowing you to work with the extracted data in your Python code.
- **Cursor Management:** Utilizes cursor objects to manage query execution and result navigation, ensuring efficient database interaction.
- **Error Handling:** Implements mechanisms to detect and handle potential errors during database operations, promoting robust and reliable applications.

OS MODULE:

The os module in Python provides a way to interact with the operating system, facilitating various operations related to file systems, processes, and environment variables.

- **os.environ:** A dictionary-like object holding the environment variables, enabling access to system configuration.

FUNCTIONS USED

A. Addstock () :

- Add stock to the product table
- Incase new product added to the product table then stock directly set to default 0.

B. Output () :

- Display the data of the required table

C. Enterval () :

- Insert new product into the product table.
- To generate a new product ID a new column (autoinc) created which autoincrements. The new product ID is Product category id concatenated with the autoincremented id. For eg:[Pcategory id=35, autoinc value=5, ProductID=355]
- Fetch the created product id from the product table and then update the stock table with the new product.

D. Placeorder () :

- Place order for a particular product
- Customer insert the product id to place order [if stock available]
- Inserted Order to order table
- Inserted dDelivery details to delivery table
- Stock updated by subtracting the quantity from existing stock

E. Delproduct () :

- Delete a specific product by entering the product ID

F. Login () :

- Login for customer and admin
- Admin id can be set through the code.
- If the id matches with the admin id then automatically admin page is open. Else customer page is open.
- If user id does not exist then ask to register
- If password wrong then ask to retry
- If userid matches with password then login successful

G. Regis () :

- Registering new user
- If username exist then ask to enter a different username
- Check if the password is strong using the passcheck() function. If password is weak then ask to enter a different password
- Insert the username and password into the customer table

H. Passcheck () :

- Check if the password is strong.
- If the password is weak then continue to ask for new password

I. Book () :

- Book a pickup for recyclable clothing
- Acquire customer data from customer table
- Insert the pickup data into the pickup table

WORKING DESCRIPTION

1. SYSTEM REQUIREMENTS:

- a) **Operating System** – Windows 10 or later, any other equivalent
- b) **Python 3.9 and later**
- c) **MySQL Community version 8.0 or later**
- d) **Python IDLE/Microsoft VS Code/Any Python IDE**

2. Database:

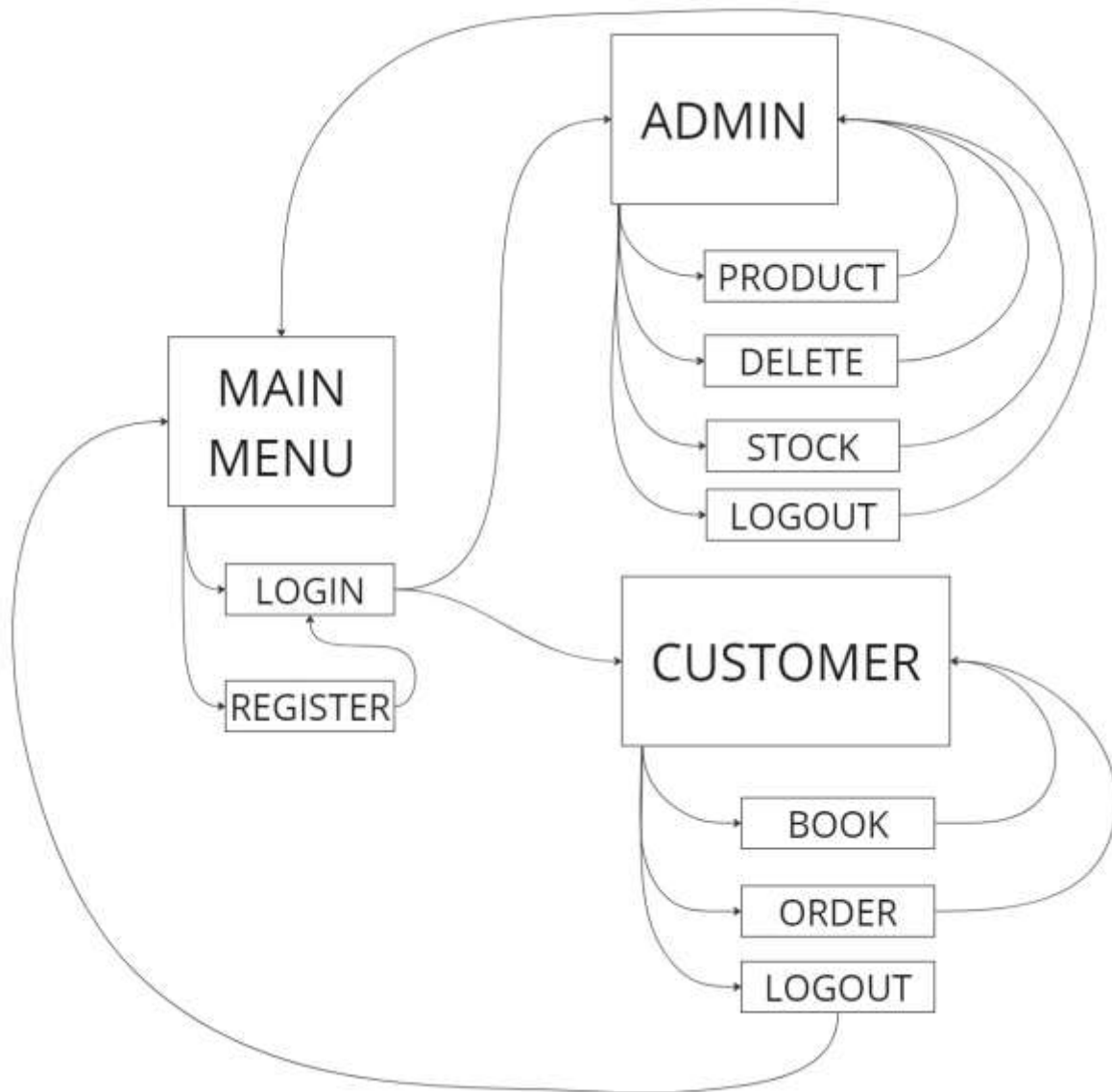
- a) Create the database and tables by executing **tables.py** file
- b) Open all the python files and **change the password** to your system sql password
- c)

3. File arrangement:

Arrange the files in the following arrangement



SUMMARY



CODE

MAIN.PY

```
import mysql.connector

import tkinter as tk

import subprocess

mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")

cs = mydb.cursor()

cs.execute("use shoppingportal")

#LOGIN

def logincli():

    root.destroy() # close this window and open below mentioned
python program using subprocesss

    subprocess.run(["python", "Login.py"])

#REGISTRATION

def regiscli():

    root.destroy()

    subprocess.run(["python", "Register.py"])

#GUI

#VARIABLES

window_width = 1550
```

```
window_height = 1000

wcolour= "#f5bae5"

wfont='comic sans ms'

#OPENING WINDOW

root = tk.Tk()

root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),
root.winfo_screenheight()))

root.title("Login/Register")

#CANVAS AND FRAME

Canvas = tk.Canvas(root, width=window_width,
height=window_height, bg=wcolour) # Set canvas background to
pink

canvas.pack()

frame=tk.Frame(root,bg=wcolour)

frame.pack(expand=True,fill="both")

frame.place(relx=0.5,rely=0.5,anchor="center")

#LOGIN

login_button = tk.Button(frame, text="Login",font=(wfont,
20),bg="#e3f5fc", command=logincli)

login_button.pack(pady=10)

login_button.config(width=20, height=4)
```

#REGISTRATION

```
register_button = tk.Button(frame, text="Register",font=(wfont,
20),bg="#e3f5fc", command=regiscli)
```

```
register_button.pack(pady=10)
```

```
register_button.config(width=20, height=4)
```

```
root.mainloop()
```

LOGIN.PY

```
import mysql.connector
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
import subprocess
```

```
import os
```

```
#VARIABLE(TO BE USED LATER)
```

```
permanentcid = 0
```

```
adminid=50
```

```
mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")
```

```
cs = mydb.cursor()
```

```
cs.execute("use shoppingportal")
```

#LOGIN FUNCTION

```
def login():
```

```
    try:
```

```
        user = entry_user.get()
```

```
        query_user = "SELECT * FROM customer WHERE userid = %s"
```

```
        val_user = (user,)
```

```
        cs.execute(query_user, val_user)
```

```
        result_user = cs.fetchone()
```

```
        if not result_user:
```

```
            messagebox.showinfo(
```

```
                "Invalid Username", "Username does not exist. Please try  
again."
```

```
            )
```

```
            return
```

```
        while True:
```

```
            password = entry_password.get()
```

```
            sql = "SELECT * FROM customer WHERE pass = %s AND userid  
= %s"
```

```
            val = (password, user)
```

```
            cs.execute(sql, val)
```

```
            result = cs.fetchone()
```

```

if result:
    global permanentcid
    global adminid
    query = "select cid from customer where userid=%s"
    cs.execute(query, (user,))
    x = cs.fetchone()
    permanentcid = x[0]
    messagebox.showinfo("Login Successful", "Login
successful!")
    os.environ["permanentcid"] = str(permanentcid)
    if permanentcid == adminid:
        root.destroy()
        subprocess.run(["python", "Admin/AMain.py"])
    else:
        root.destroy()
        subprocess.run(["python", "Customer/CMain.py"])
    break
else:
    messagebox.showinfo("Invalid Password", "Invalid
password. Please try again.")

```

```

        return

    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")

#GUI

#VARIABLES

wsize = 40

wfontsize = 20

window_width = 1550

window_height = 1000

wcolour = "#f5bae5"

wbox = "#fce3f6"

wfont = 'comic sans ms'

#OPENING WINDOW

root = tk.Tk()

root.title("Login Page")

root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),
root.winfo_screenheight()))

#CANVAS AND FRAME

canvas = tk.Canvas(root, width=window_width,
height=window_height, bg=wcolour) # Set canvas background to
pink

```

```
canvas.pack()

frame = tk.Frame(root , bg=wcolour)

frame.pack(expand=True , fill = "both")

frame.place(relx=0.5 , rely=0.5 , anchor="center")

#USERNAME

tk.Label(frame, text="USERNAME" , font = (wfont , wfontsize) , bg=
wcolour).pack()

entry_user = tk.Entry(frame , width = wsize , font = (wfont,wfontsize)
, bg=wbox)

entry_user.pack(pady=20)

#PASSWORD

tk.Label(frame, text="PASSWORD" , font=(wfont, wfontsize),
bg=wcolour).pack()

entry_password = tk.Entry(frame , show="*" ,
width=wsize,font=(wfont , wfontsize) , bg=wbox)

entry_password.pack(pady=20)

#LOGIN BUTTON

login_button = tk.Button(frame , text="LOGIN" , width=10 ,
font=(wfont , wfontsize) , bg="#e3f5fc" , command=login)

login_button.pack(pady=50)

root.mainloop()
```

REGISTER.PY

```
import mysql.connector

import tkinter as tk

from tkinter import messagebox

import subprocess

mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")

cs = mydb.cursor()

cs.execute("use shoppingportal")

#PASSWORD CHECK FUNCTION

def passcheck(password):

    has_upper = False

    has_lower = False

    has_digit = False

    has_special = False

    has_space = False

    for char in password:

        if char.isupper():

            has_upper = True

        elif char.islower():

            has_lower = True
```

```
elif char.isdigit():
    has_digit = True
elif char in "!@#$%^&*()_+=[{}|;':,<>/?":
    has_special = True
elif char.isspace():
    has_space = True
return (
    len(password) >= 8
    and has_upper
    and has_lower
    and has_digit
    and has_special
    and not has_space
)
```

#REGISTRATION FUNCTION

```
def regiscli():
    try:
        user = entry_user.get()
        fname = entry_fname.get()
        lname = entry_lname.get()
```

```

phone = entry_phone.get()
password = entry_password.get()
while True:
    if passcheck(password):
        break
    else:
        messagebox.showinfo(
            "Weak Password",
            ""

```

Password is weak. Please consider the following guidelines:

- Use a combination of uppercase and lowercase letters
- Include at least one digit
- Add at least one special character (!@#\$%^&*()_+-= []{}|;':\".,<>/?
- Make the password at least 8 characters long
- Do not use spaces in the password""",
)

```

return

query = "SELECT * FROM customer WHERE userid = %s"
val = (user,)
cs.execute(query, val)

```

```

result = cs.fetchone()

if result:

    messagebox.showinfo(

        "Username Exists", "Username already exists. Please try a
different one."

    )

    return

    query = "INSERT INTO customer (firstname, lastname, phone,
pass, userid) VALUES (%s, %s, %s, %s, %s)"
    val = (fname, lname, phone, password, user)
    cs.execute(query, val)
    mydb.commit()

    messagebox.showinfo("Registration      Successful",      "User
registered successfully!")

    root.destroy()

    subprocess.run(["python", "Main.py"])

except Exception as e:

    messagebox.showerror("Error", f"An error occurred: {e}")

#GUI

#VARIABLES

wsize=40

```

```
wfontsize=20

window_width = 1550
window_height = 1000
wcolour= "#f5bae5"
wbox="#fce3f6"
wfont='comic sans ms'

#OPENING WINDOW

root = tk.Tk()

root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),
root.winfo_screenheight()))

#CANVAS AND FRAME

canvas = tk.Canvas(root, width=window_width,
height=window_height, bg=wcolour) # Set canvas background to
pink

canvas.pack()

frame=tk.Frame(root,bg=wcolour)

frame.pack(expand=True,fill="both")

frame.place(relx=0.5,rely=0.5,anchor="center")

#USERNAME

tk.Label(frame, text="USERNAME", font=(wfont, wfontsize),
bg=wcolour).pack()
```

```
entry_user = tk.Entry(frame,width=wsizer,font=(wfont,
wfontsize),bg=wbox)
```

```
entry_user.pack(pady=20)
```

```
#FIRST NAME
```

```
tk.Label(frame, text="FIRST NAME", font=(wfont, wfontsize),
bg=wcolour).pack()
```

```
entry_fname = tk.Entry(frame,width=wsizer,font=(wfont,
wfontsize,),bg=wbox)
```

```
entry_fname.pack(pady=20)
```

```
#LAST NAME
```

```
tk.Label(frame, text="LAST NAME", font=(wfont, wfontsize),
bg=wcolour).pack()
```

```
entry_lname = tk.Entry(frame,width=wsizer,font=(wfont,
wfontsize),bg=wbox)
```

```
entry_lname.pack(pady=20)
```

```
#PHONE NO
```

```
tk.Label(frame, text="PHONE", font=(wfont, wfontsize),
bg=wcolour).pack()
```

```
entry_phone = tk.Entry(frame,width=wsizer,font=(wfont,
wfontsize),bg=wbox)
```

```
entry_phone.pack(pady=20)
```

```
#PASSWORD
```

```
tk.Label(frame, text="PASSWORD", font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_password =
```

```
tk.Entry(frame,show="*",width=wsizer,font=(wfont,wfontsize),bg=w  
box)
```

```
entry_password.pack(pady=20)
```

```
#REGISTER BUTTON
```

```
register_button = tk.Button(frame, text="REGISTER",font=(wfont,  
20),bg="#e3f5fc", command=regiscli)
```

```
register_button.pack(pady=80)
```

```
root.mainloop()
```

AMAIN.PY

```
import mysql.connector
```

```
import tkinter as tk
```

```
import subprocess
```

```
mydb = mysql.connector.connect(host="localhost", user="root",  
password="123456")
```

```
cs = mydb.cursor()
```

```
cs.execute("use shoppingportal")
```

```
def add_stock():
```

```
    root.destroy()
```

```
subprocess.run(["python", "Admin/Stock.py"])

def enter_val():
    root.destroy()
    subprocess.run(["python", "Admin/Product.py"])

def del_product():
    root.destroy()
    subprocess.run(["python", "Admin/Delete.py"])

def logout():
    root.destroy()
    subprocess.run(["python", "Main.py"])

#variables

window_width = 1550
window_height = 1000
wcolour= "#f5bae5"
wfont="comic sans ms"

#OPENING WINDOW

root = tk.Tk()

root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),
root.winfo_screenheight()))

root.title("ADMIN ACCESS")
```

#CANVAS AND FRAME

```
canvas = tk.Canvas(root, width=window_width,  
height=window_height, bg=wcolour)
```

```
canvas.pack()
```

```
frame=tk.Frame(root,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

#BOOK BUTTON

```
enterval_button =
```

```
tk.Button(frame, text="ADD PRODUCT",font=(wfont,  
20),bg="#e3f5fc", command=enter_val)
```

```
enterval_button.pack(pady=20)
```

```
enterval_button.config(width=20, height=2)
```

#DELETE PRODUCT

```
delval_button = tk.Button(frame, text="DELETE  
PRODUCT",font=(wfont, 20),bg="#e3f5fc", command=del_product)
```

```
delval_button.pack(pady=20)
```

```
delval_button.config(width=20, height=2)
```

#UPDATE STOCK

```
delval_button = tk.Button(frame, text="UPDATE  
STOCK",font=(wfont, 20),bg="#e3f5fc", command=add_stock)
```

```
delval_button.pack(pady=20)
```

```
delval_button.config(width=20, height=2)
```

```
#LOGOUT
```

```
delval_button = tk.Button(frame, text="LOGOUT",font=(wfont,  
20),bg="#e3f5fc", command=logout)
```

```
delval_button.pack(pady=20)
```

```
delval_button.config(width=20, height=2)
```

```
root.mainloop()
```

DELETE.PY

```
import mysql.connector
```

```
import tkinter as tk
```

```
import subprocess
```

```
from tkinter import messagebox
```

```
mydb = mysql.connector.connect(host="localhost", user="root",  
password="123456")
```

```
cs = mydb.cursor()
```

```
cs.execute("use shoppingportal")
```

```
#DELETE PRODUCT FUNCTION
```

```
def delproduct():
```

```
    try:
```

```
        product_id = entry_product_id.get()
```

```
        query = "SELECT * FROM product WHERE itemcode = %s"
```

```

val = (product_id,)
cs.execute(query, val)
result = cs.fetchone()
if not result:
    messagebox.showinfo("Error", "Product does not exist")
else:
    query = "DELETE FROM product WHERE itemcode = %s"
    cs.execute(query, (product_id,))
    query = "DELETE FROM stock WHERE itemcode = %s"
    cs.execute(query, (product_id,))
    mydb.commit()
    messagebox.showinfo("Product Deleted", "Product deleted
successful!")
    root.destroy()
    subprocess.run(["python", "Admin/AMain.py"])
except Exception as e:
    messagebox.showerror("Error", f"An error occurred: {e}")
#BACK NAVIGATION FUNCTION
def back():
    root.destroy()
    subprocess.run(["python", "Admin/AMain.py"])

```

```
#GUI
```

```
#VARIABLES
```

```
wsiz=40
```

```
wfonsiz=15
```

```
window_width = 1550
```

```
window_height = 1000
```

```
wcolour= "#f5bae5"
```

```
wfont="comic sans ms"
```

```
wbox="#fce3f6"
```

```
#OPENING WINDOW
```

```
root = tk.Tk()
```

```
root.title("Delete Product")
```

```
root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),  
root.winfo_screenheight()))
```

```
#CANVAS AND FRAME
```

```
canvas = tk.Canvas(root, width=window_width,  
height=window_height, bg=wcolour)
```

```
canvas.pack()
```

```
frame=tk.Frame(root,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

```
#PRODUCT ID
```

```
tk.Label(frame, text="ENTER PRODUCT ID",width=wsizer,
font=(wfont, wfontsize), bg=wcolour).pack()
```

```
entry_product_id
```

```
=
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_product_id.pack(pady=20)
```

```
#DELETE PRODUCT
```

```
delete_product_button = tk.Button(frame, text="DELETE
PRODUCT",width=20,font=(wfont,wfontsize),bg=wbox,
command=delproduct)
```

```
delete_product_button.pack(pady=50)
```

```
#BACK BUTTON
```

```
navback = tk.Button(frame, text="BACK",font=(wfont,
20),bg="#e3f5fc", command=back)
```

```
navback.pack(pady=20)
```

```
navback.config(width=5, height=1)
```

```
root.mainloop()
```

PRODUCT.PY

```
import mysql.connector
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
import subprocess
```

```

mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")

cs = mydb.cursor()

cs.execute("use shoppingportal")

#ADDING PRODUCT FUNCTION

def enterval():

    try:

        x = entry_category.get()

        cs.execute("SELECT typeid FROM pcategory WHERE type=%s",
(x,))

        tid = cs.fetchone()

        if tid:

            category = x

            gender = entry_gender.get()

            size = entry_size.get()

            price = entry_price.get()

            # inserting into the product table

            query = "INSERT INTO product(typeid, category, gender, size,
price) VALUES (%s, %s, %s, %s, %s)"

            val = (tid[0], category, gender, size, price)

            cs.execute(query, val)

```

```

mydb.commit()

cs.execute("UPDATE    product    SET    itemcode    =
CONCAT(typeid,autoinc)")

cs.execute(

    "SELECT itemcode FROM product WHERE autoinc = (SELECT
MAX(autoinc) FROM product)"

)

pid = cs.fetchone()

# inserting into the stock table

query2 = "INSERT INTO stock(itemcode, stock) VALUES (%s,
%s)"

val2 = (pid[0], 0)

cs.execute(query2, val2)

mydb.commit()

messagebox.showinfo("Product  Added", "Product  Added
successfully!")

root.destroy()

subprocess.run(["python", "Admin/AMain.py"])

except Exception as e:

    messagebox.showerror("Error", f"An error occurred: {e}")

```

```
#BACK
```

```
def back():
```

```
    root.destroy()
```

```
    subprocess.run(["python", "Admin/AMain.py"])
```

```
#GUI
```

```
#VARIABLES
```

```
wsiz=40
```

```
wfonsiz=15
```

```
window_width = 1550
```

```
window_height = 1000
```

```
wcolour= "#f5bae5"
```

```
wfont="comic sans ms"
```

```
wbox="#fce3f6"
```

```
#OPENING WINDOW
```

```
root = tk.Tk()
```

```
root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),  
root.winfo_screenheight()))
```

```
root.title("Add Product Details")
```

#CANVAS AND FRAME

```
canvas = tk.Canvas(root, width=window_width,  
height=window_height, bg=wcolour) # Set canvas background to  
pink
```

```
canvas.pack()
```

```
frame=tk.Frame(root,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

#CATEGORY

```
tk.Label(frame, text="ENTER CATEGORY",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_category = tk.StringVar()
```

```
entry_category.set("Choose") # Default value
```

```
cat_menu = tk.OptionMenu(frame,entry_category,  
"SHIRT","TROUSER","SHOES","ACCESSORY","HOUSEHOLD")
```

```
cat_menu.pack(pady=10)
```

#GENDER

```
tk.Label(frame, text="ENTER GENDER",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_gender =  
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_gender.pack(pady=10)
```

```
#SIZE
```

```
tk.Label(frame, text="ENTER SIZE",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_size
```

```
=
```

```
tk.Entry(frame,width=wsizel,font=(wfont,wfontsize),bg=wbox)
```

```
entry_size.pack(pady=10)
```

```
#PRICE
```

```
tk.Label(frame, text="ENTER PRICE",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_price =
```

```
tk.Entry(frame,width=wsizel,font=(wfont,wfontsize),bg=wbox)
```

```
entry_price.pack(pady=10)
```

```
#ADD PRODUCT
```

```
enter_value_button = tk.Button(frame, text="ADD  
PRODUCT",font=(wfont, 20),bg="#e3f5fc", command=entervall)
```

```
enter_value_button.pack(pady=70)
```

```
#back button
```

```
navback = tk.Button(frame, text="BACK",font=(wfont,  
20),bg="#e3f5fc", command=back)
```

```
navback.pack(pady=20)
```

```
navback.config(width=5, height=1)
```

```
root.mainloop()
```

STOCK.PY

```
import mysql.connector

import tkinter as tk

from tkinter import messagebox

import subprocess

mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")

cs = mydb.cursor()

cs.execute("use shoppingportal")

#ADDING STOCK FUNCTION

def addstock():

    try:

        item_code = entry_item_code.get()

        stock = entry_stock.get()

        # check if the item code exists in the stock table

        cs.execute("SELECT * FROM stock WHERE itemcode = %s",
(item_code,))

        existing_stock = cs.fetchone()

        if existing_stock:

            # item code exists, updates the stock
```

```
query = "UPDATE stock SET stock = stock + %s WHERE  
itemcode = %s"
```

```
val = (stock, item_code)
```

```
cs.execute(query, val)
```

```
else:
```

```
# item code does not exist, insert a new record
```

```
query = "INSERT INTO stock (itemcode, stock) VALUES (%s,  
%s)"
```

```
val = (item_code, stock)
```

```
cs.execute(query, val)
```

```
mydb.commit()
```

```
messagebox.showinfo("Stock Added", "Stock updated  
successfully!")
```

```
root.destroy()
```

```
subprocess.run(["python", "Admin/Amain.py"])
```

```
except Exception as e:
```

```
messagebox.showerror("Error", f"An error occurred: {e}")
```

```
#BACK NAVIGATION FUNCTION
```

```
def back():
```

```
root.destroy()
```

```
subprocess.run(["python", "Admin/AMain.py"])
```

```
#GUI
```

```
#OPENING WINDOW
```

```
root = tk.Tk()
```

```
root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),  
root.winfo_screenheight()))
```

```
root.title("Add Stock")
```

```
#VARIABLES
```

```
wsizer=40
```

```
wfontsize=15
```

```
window_width = 1550
```

```
window_height = 1000
```

```
wcolour= "#f5bae5"
```

```
wfont="comic sans ms"
```

```
wbox="#fce3f6"
```

```
#CANVAS AND FRAME
```

```
canvas = tk.Canvas(root, width=window_width,  
height=window_height, bg=wcolour) # Set canvas background to  
pink
```

```
canvas.pack()
```

```
frame=tk.Frame(root,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

```
#ITEMCODE
```

```
tk.Label(frame, text="ENTER ITEMCODE",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_item_code =
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_item_code.pack(pady=20)
```

```
#STOCK
```

```
tk.Label(frame, text="ENTER STOCK",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_stock =
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_stock.pack(pady=20)
```

```
#ADD STOCK
```

```
add_stock_button = tk.Button(frame, text="Add Stock",font=(wfont,  
20),bg="#e3f5fc", command=addstock)
```

```
add_stock_button.pack(pady=50)
```

```
#BACK
```

```
navback = tk.Button(frame, text="BACK",font=(wfont,  
20),bg="#e3f5fc", command=back)
```

```
navback.pack(pady=20)
```

```
navback.config(width=5, height=1)
```

```
root.mainloop()
```

CMAIN.PY

```
import mysql.connector
```

```
from datetime import datetime
```

```
import tkinter as tk
```

```
import subprocess
```

```
mydb = mysql.connector.connect(host="localhost", user="root",  
password="123456")
```

```
cs = mydb.cursor()
```

```
date = datetime.now()
```

```
cs.execute("use shoppingportal")
```

```
#BOOK A PICKUP
```

```
def bookcli():
```

```
    root.destroy()
```

```
    subprocess.run(["python", "Customer/Book.py"])
```

```
#ORDER
```

```

def place_order():
    root.destroy()
    subprocess.run(["python", "Customer/Order.py"])
#LOGOUT
def logout():
    root.destroy()
    subprocess.run(["python", "Main.py"])
#GUI
#VARIABLES
window_width = 1550
window_height = 1000
wcolour= "#f5bae5"
wfont="comic sans ms"
#OPENING WINDOW
root = tk.Tk()
root.title("THREADSORAMA")
#CANVAS AND FRAME
canvas = tk.Canvas(root, width=window_width,
height=window_height, bg=wcolour) # Set canvas background to
pink
canvas.pack()

```

```
frame=tk.Frame(root,bg=wcolour)
frame.pack(expand=True,fill="both")
frame.place(relx=0.5,rely=0.5,anchor="center")

#BOOK PICKUP

book_button = tk.Button(frame, text="BOOK A PICKUP",font=(wfont,
20),bg="#e3f5fc", command=bookcli)

book_button.pack(pady=10)

book_button.config(width=20, height=4)

#PLACE ORDER

order_button = tk.Button(frame, text="ORDER",font=(wfont,
20),bg="#e3f5fc", command=place_order)

order_button.pack(pady=20)

order_button.config(width=20, height=4)

#LOGOUT

delval_button = tk.Button(frame, text="LOGOUT",font=(wfont,
20),bg="#e3f5fc", command=logout)

delval_button.pack(pady=20)

delval_button.config(width=20, height=4)

root.mainloop()
```

BOOK.PY

```
import subprocess

import tkinter as tk

from tkinter import messagebox

import os

import mysql.connector

mydb = mysql.connector.connect(host="localhost", user="root",
password="123456")

cs = mydb.cursor()

permanentcid = os.environ.get("permanentcid")

cs.execute("use shoppingportal")

#BOOK FUNCTION

def book():

    try:

        global permanentcid

        mat = entry_material.get()

        w = entry_weight.get()

        mcondition = condition_var.get()

        sadd = entry_street_address.get()

        pcode = entry_postal_code.get()

        town = entry_town.get()
```

```
query = "INSERT INTO pickupportal VALUES (%s, %s, %s, %s,  
%s, %s, %s)"
```

```
val = (mat, w, mcondition, permanentcid, sadd, pcode, town)
```

```
cs.execute(query, val)
```

```
mydb.commit()
```

```
messagebox.showinfo("Booking Successful", "Booking  
successful!")
```

```
root.destroy()
```

```
subprocess.run(["python", "Customer/Cmain.py"])
```

```
except Exception as e:
```

```
messagebox.showerror("Error", f"An error occurred: {e}")
```

```
#BACK FUNCTION
```

```
def back():
```

```
    root.destroy()
```

```
    subprocess.run(["python", "Customer/CMain.py"])
```

```
#GUI
```

```
#VARIABLES
```

```
wsizer=40
```

```
wfontsize=15
```

```
window_width = 1550
```

```
window_height = 1000
```

```
wcolour= "#f5bae5"
```

```
wfont="comic sans ms"
```

```
wbox="#fce3f6"
```

```
#OPENING WINDOW
```

```
root = tk.Tk()
```

```
root.title("Booking")
```

```
root.geometry("{0}x{1}+0+0".format(root.winfo_screenwidth(),  
root.winfo_screenheight()))
```

```
root.title("BOOK PICKUP")
```

```
#CANVAS AND FRAME
```

```
canvas = tk.Canvas(root, width=window_width,  
height=window_height, bg=wcolour) # Set canvas background to  
pink
```

```
canvas.pack()
```

```
frame=tk.Frame(root,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

```
#MATERIAL
```

```
tk.Label(frame, text="Choose Material:",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```

entry_material                                     =
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)

entry_material.pack(pady=10)

#WEIGHT

tk.Label(frame, text="Enter Weight:",font=(wfont, wfontsize),
bg=wcolour).pack()

entry_weight                                       =
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)

entry_weight.pack(pady=10)

#CONDITION

tk.Label(frame, text="Enter Condition:",font=(wfont, wfontsize),
bg=wcolour).pack()

condition_var = tk.StringVar()

condition_var.set("Choose") # Default value

condition_menu = tk.OptionMenu(frame,condition_var, "Worn out",
"Average", "Brand New")

condition_menu.pack(pady=10)

#STREET ADDRESS

tk.Label(frame, text="Enter Street Address:",font=(wfont, wfontsize),
bg=wcolour).pack()

entry_street_address                               =
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)

entry_street_address.pack(pady=10)

```

```
#POSTAL CODE
```

```
tk.Label(frame, text="Enter Postal Code:",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_postal_code =
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_postal_code.pack(pady=10)
```

```
#TOWN
```

```
tk.Label(frame, text="Enter Town:",font=(wfont, wfontsize),  
bg=wcolour).pack()
```

```
entry_town
```

```
=
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_town.pack(pady=10)
```

```
#BOOK
```

```
book_button = tk.Button(frame, text="Book  
Pickup",width=10,font=(wfont,wfontsize),bg="#e3f5fc",  
command=book)
```

```
book_button.pack(pady=10)
```

```
#BACK
```

```
navback = tk.Button(frame, text="BACK",font=(wfont,  
20),bg="#e3f5fc", command=back)
```

```
navback.pack(pady=20)
```

```
navback.config(width=5, height=1)
```

```
root.mainloop()
```

ORDER.PY

```
import mysql.connector
from datetime import date
import tkinter as tk
from tkinter import messagebox, Entry
import os
import subprocess

permanentcid = os.environ.get("permanentcid")

mydb = mysql.connector.connect(
    host="localhost", user="root", password="123456",
    database="shoppingportal"
)

cs = mydb.cursor()

#VARIABLES

wsize=40

wfontsize=15

window_width = 1550

window_height = 1000

wcolour= "#f5bae5"

wfont="comic sans ms"

wbox="#fce3f6"
```

#OPENING WINDOW

```
my_w = tk.Tk()
```

```
my_w.geometry("{0}x{1}+0+0".format(my_w.winfo_screenwidth(),  
my_w.winfo_screenheight()))
```

```
my_w.title("Shopping Portal")
```

#CANVAS AND FRAME

```
canvas = tk.Canvas(my_w, width=window_width,  
height=window_height, bg=wcolour) # Set canvas background to  
pink
```

```
canvas.pack()
```

```
frame=tk.Frame(my_w,bg=wcolour)
```

```
frame.pack(expand=True,fill="both")
```

```
frame.place(relx=0.5,rely=0.5,anchor="center")
```

#STOCK DISPLAY

```
cs.execute("SELECT  
p.itemcode,p.category,p.gender,p.size,p.price,s.stock FROM product  
p JOIN stock s ON p.itemcode = s.itemcode;")
```

```
stock_column_names = [desc[0] for desc in cs.description]
```

```
#CREATING LABELS(PRODUCT ID AND STOCK
```

```
for j in range(len(stock_column_names)):
```

```
    label = tk.Label(frame, text=stock_column_names[j].upper(),  
font=(wfont, 20),bg=wcolour)
```

```
    label.grid(row=0, column=j)
```

```
i = 1
```

```
for stock_data in cs:
```

```
    for j in range(len(stock_data)):
```

```
        e = Entry(frame, width=10, fg="green", font=(wfont, 12))
```

```
        e.grid(row=i, column=j)
```

```
        e.insert(tk.END, stock_data[j])
```

```
    i += 1
```

```
#ORDER FUNCTION
```

```
def placingorder():
```

```
    try:
```

```
        # working
```

```
        global permanentcid
```

```
        cid = str(permanentcid)
```

```
        today = date.today()
```

```
        ic = str(entry_product_id.get())
```

```
        qt = str(entry_quantity.get())
```

```

dop = str(today.strftime("20%y-%m-%d"))
sadd = str(entry_shipping_address.get())
pcode = str(entry_postal_code.get())
town = str(entry_town.get())

# working

query = "INSERT INTO orderdetails (customerid, itemcode,
quantity, dateofpurchase, shippingaddress) VALUES (%s, %s, %s, %s,
%s)"

val = (cid, ic, qt, dop, sadd)
cs.execute(query, val)
mydb.commit()

# not working

cs.execute("SELECT max(orderid) from orderdetails")
x = cs.fetchone()
oid = x[0]

# this is working

query = "INSERT INTO deliverysystem VALUES (%s, %s, %s,
%s)"

val = (str(oid), str(cid), str(pcode), str(town))
cs.execute(query, val)
mydb.commit()

```

```
query2 = "UPDATE stock SET stock = stock - %s WHERE itemcode  
= %s"
```

```
val2 = (str(qt), str(ic))
```

```
cs.execute(query2, val2)
```

```
mydb.commit()
```

```
messagebox.showinfo("Order Placed", "Order placed  
successfully!")
```

```
my_w.destroy()
```

```
subprocess.run(["python", "Customer/Cmain.py"])
```

```
except Exception as e:
```

```
messagebox.showerror("Error", f"An error occurred: {e}")
```

```
#BACK NAVIGATION
```

```
def back():
```

```
my_w.destroy()
```

```
subprocess.run(["python", "Customer/CMain.py"])
```

```
#PRODUCT ID
```

```
tk.Label(frame, text="Enter Product ID:",font=(wfont, wfontsize),  
bg=wcolour).grid(row=i, column=0)
```

```
entry_product_id =
```

```
tk.Entry(frame,width=wsizer,font=(wfont,wfontsize),bg=wbox)
```

```
entry_product_id.grid(row=i, column=1,pady=5)
```

```
i += 1
```

#QUANTITY

```
tk.Label(frame, text="Enter Quantity:",font=(wfont, wfontsize),  
bg=wcolour).grid(row=i, column=0)
```

```
entry_quantity =
```

```
tk.Entry(frame,width=wsize,font=(wfont,wfontsize),bg=wbox)
```

```
entry_quantity.grid(row=i, column=1,pady=5)
```

```
i += 1
```

#SHIPPING ADDRESS

```
tk.Label(frame, text="Enter Shipping Address:",font=(wfont,  
wfontsize), bg=wcolour).grid(row=i, column=0)
```

```
entry_shipping_address =
```

```
tk.Entry(frame,width=wsize,font=(wfont,wfontsize),bg=wbox)
```

```
entry_shipping_address.grid(row=i, column=1,pady=5)
```

```
i += 1
```

#POSTAL CODE

```
tk.Label(frame, text="Enter Postal Code:",font=(wfont, wfontsize),  
bg=wcolour).grid(row=i, column=0)
```

```
entry_postal_code =
```

```
tk.Entry(frame,width=wsize,font=(wfont,wfontsize),bg=wbox)
```

```
entry_postal_code.grid(row=i, column=1,pady=5)
```

```
i += 1
```

```
#TOWN
```

```
tk.Label(frame, text="Enter Town:",font=(wfont, wfontsize),  
bg=wcolour).grid(row=i, column=0)
```

```
entry_town =
```

```
tk.Entry(frame,width=wsiz,font=(wfont,wfontsize),bg=wbox)
```

```
entry_town.grid(row=i, column=1,pady=5)
```

```
i += 1
```

```
#PLACE ORDER BUTTON
```

```
place_order_button =
```

```
tk.Button(frame, text="Place Order",font=(wfont, 20),bg="#e3f5fc",  
command=placingorder)
```

```
place_order_button.grid(row=i, column=0, columnspan=2, pady=20)
```

```
i+=1
```

```
#BACK BUTTON
```

```
navback =
```

```
tk.Button(frame, text="BACK",font=(wfont, 20),bg="#e3f5fc",  
command=back)
```

```
navback.grid(row=i, column=0, columnspan=2, pady=5)
```

```
my_w.mainloop()
```

Tables.py

```
import mysql.connector

x=input("Enter your password:")

mydb =
mysql.connector.connect(host="localhost",user="root",password=st
r(x))

cs=mydb.cursor()

def db():

    cs.execute("create database shoppingportal")

    cs.execute("use shoppingportal")

def tables():

    query1=("Create table product (typeid int references
pcategory(typeid) ,itemcode integer(3) unique,category
varchar(30),gender varchar(1) ,size varchar(3),price integer(4) not
null,autoinc int(40) auto_increment primary key);")

    query2=("Create table orderdetails (orderid integer(3)
auto_increment primary key unique not null,customerid integer(3)
references customerdetails(CID),itemcode integer(3) references
product(itemcode),quantity integer(2) not null,dateofpurchase
date,shippingaddress varchar(50) not null);")

    query3=("Create table stock (itemcode integer(3) references
product(itemcode),stock integer(5));")

    query4=("CREATE TABLE pickupportal (material
VARCHAR(30),weight DECIMAL(10,2),materialcondition
```

```
VARCHAR(10),cid INT(3) REFERENCES customer(cid),streetaddress  
VARCHAR(50),postalcode VARCHAR(6),town VARCHAR(20));")
```

```
query5=("Create table deliverysystem(orderid int references  
orderdetails(orderid),cid int references customer(cid),postalcode  
int(6), town varchar(20));")
```

```
query6=("CREATE TABLE customer(cid integer(3) primary key  
auto_increment,firstname varchar(20), lastname varchar(20), phone  
integer(10) unique, pass VARCHAR(50) NOT NULL,userid  
varchar(30) unique);")
```

```
query7=("Create table pcategory(typeid integer(2) not null  
primary key,type varchar(30))")
```

```
query21=("Insert                into                pcategory  
values(35,'shirt'),(36,'trouser'),(37,'shoes'),(38,'accessory'),(39,'hou  
sehold');")
```

```
cs.execute(query1)
```

```
print("Product table created successfully")
```

```
cs.execute(query2)
```

```
print("Order Details table created successfully")
```

```
cs.execute(query3)
```

```
print("Stock table created successfully")
```

```
cs.execute(query4)
```

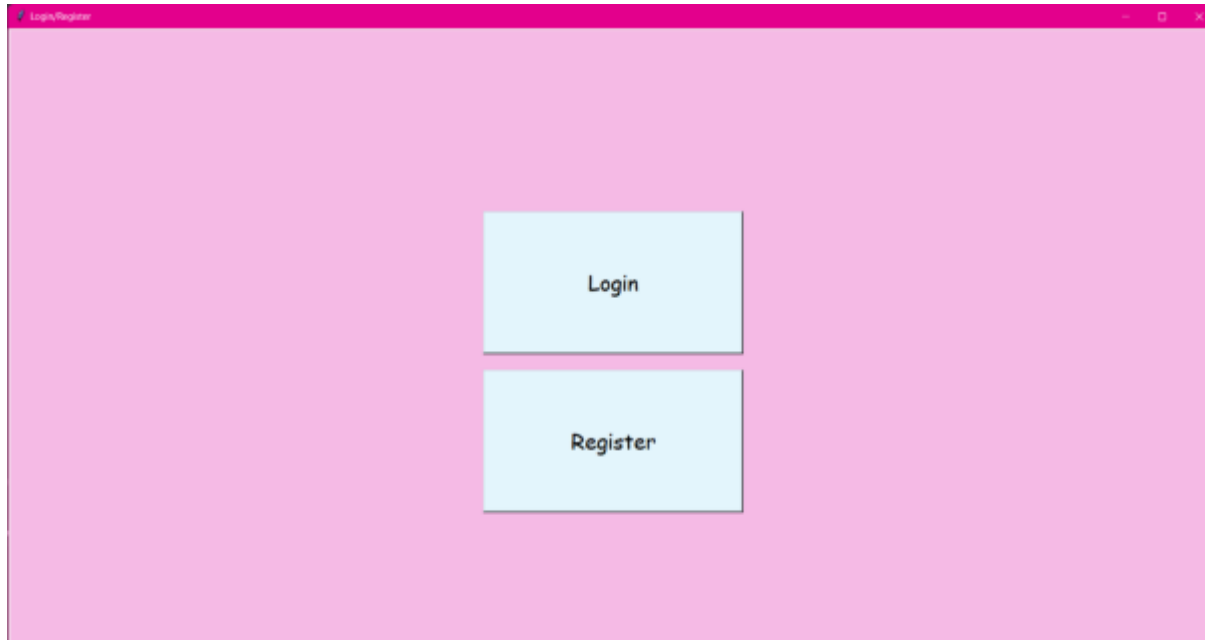
```
print("Pickup Details table created successfully")
```

```
cs.execute(query5)
```

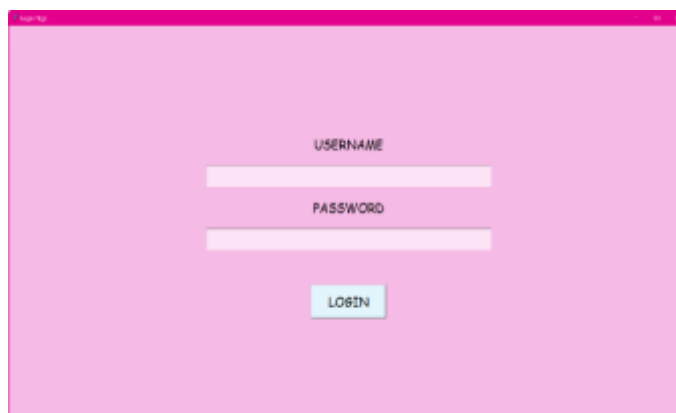
```
print("Delivery Details table created successfully")
cs.execute(query6)
print("Customer Details table created successfully")
cs.execute(query7)
print("Product category table created successfully")
cs.execute(query21)
print("Category added to table successfully")
x=int(input("
Do you want to create the tables?
1:Yes
2:No
"))
if x==1:
    db()
    tables()
elif x==2:
    cs.execute("use shoppingportal")
    print("Using shoppingportal")
```

OUTPUT

MAIN.PY



REGISTER.PY



```
mysql> select * from customer;
+----+-----+-----+-----+-----+-----+
| cid | firstname | lastname | phone | pass | userid |
+----+-----+-----+-----+-----+-----+
| 20 | testcustomer | test | 123456769 | c | c |
| 50 | test | test | 129456789 | i | i |
| 65 | ADAM | EVE | 123456289 | @Pass123 | USER2 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

LOGIN.PY

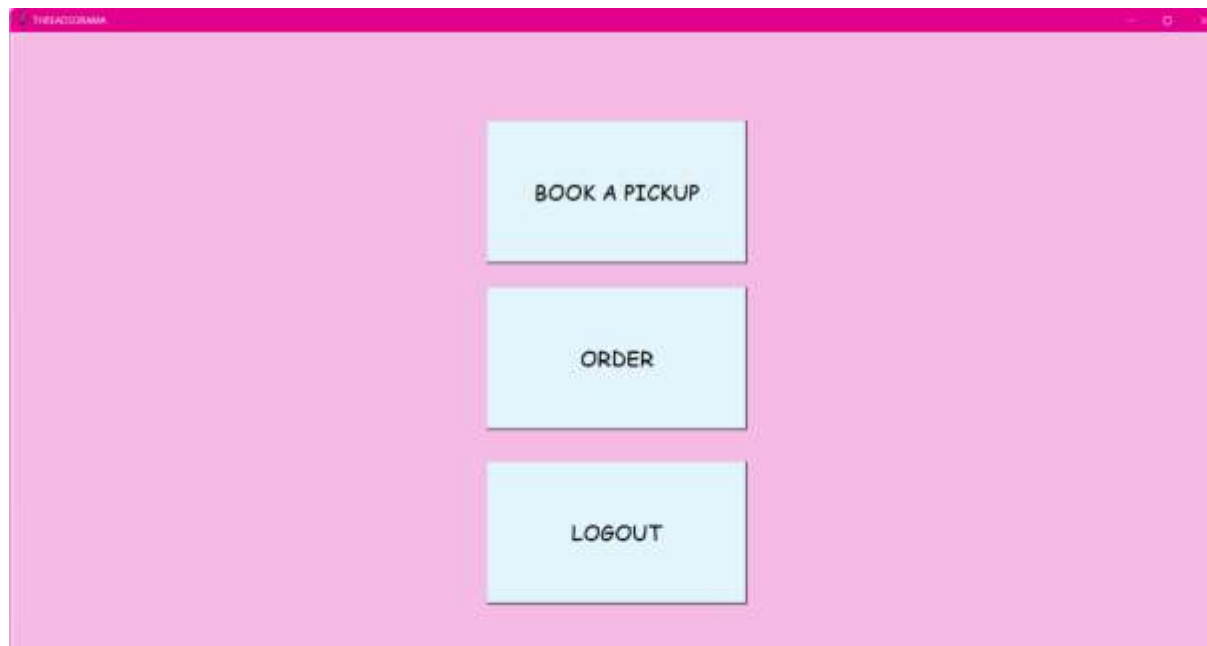


A registration form with a pink background. It contains six white input fields stacked vertically, labeled USERNAME, FIRST NAME, LAST NAME, PHONE, and PASSWORD. Below the fields is a light blue button labeled REGISTER.



A login form with a pink background. It contains two white input fields, one for USERNAME and one for PASSWORD. Below the fields is a light blue button labeled LOGIN. To the right of the fields is a small icon of a smartphone displaying a login screen.

Cmain.py



A main menu window titled 'THEATERMAMA' with a pink background. It features three light blue buttons stacked vertically in the center, labeled BOOK A PICKUP, ORDER, and LOGOUT.

Book.py

(Please Select)

Enter Weight

Enter Condition

Enter Street Address

Enter Postal Code

Enter Town

Book Policy

BACK

Choose Material

Enter Weight

Enter Condition

Enter Street Address

Enter Postal Code

Enter Town

Book Policy

BACK

```
mysql> select * from pickupportal;
```

material	weight	materialcondition	cid	streetaddress	postalcode	town
adf	3.00	Average	28	adf	123456	a
nylon	20.00	Worn out	28	xyz colony	123456	delhi

2 rows in set (0.00 sec)

Order.py

ITEMCODE

CATEGORY

GENDER SIZE PRICE STOCK

Enter Product ID

Enter Quantity

Enter Shipping Address

Enter Postal Code

Enter Town

Place Order

BACK

ITEMCODE

CATEGORY

GENDER SIZE PRICE STOCK

Enter Product ID

Enter Quantity

Enter Shipping Address

Enter Postal Code

Enter Town

Place Order

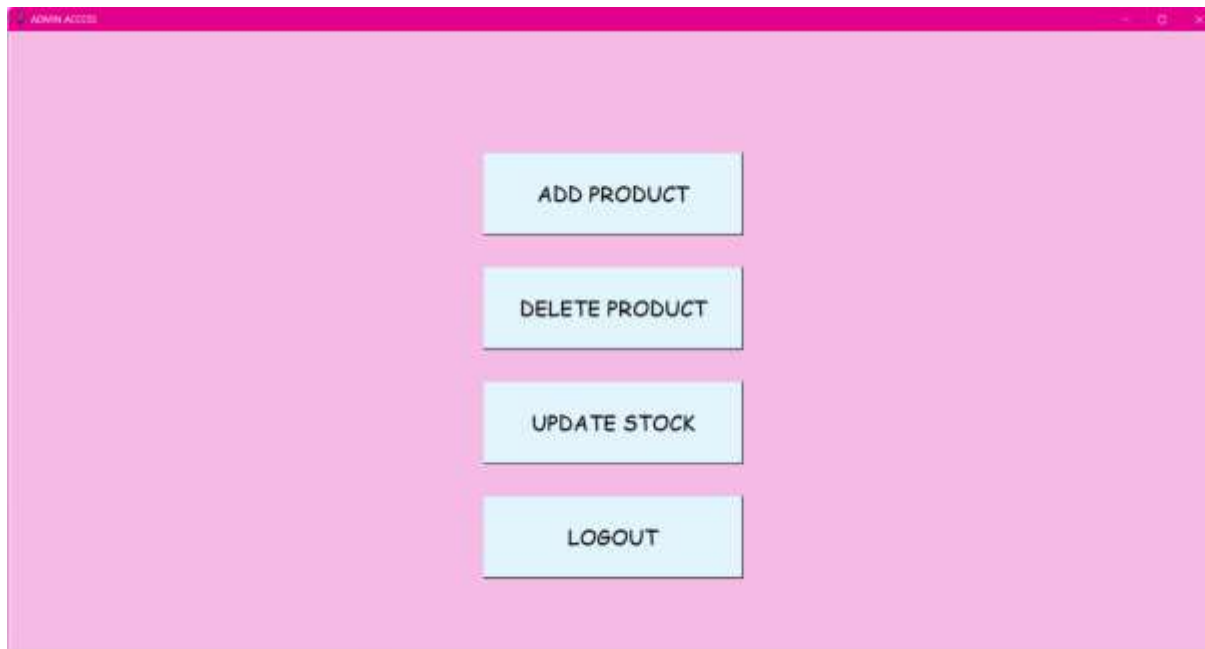
BACK

```
mysql> select * from orderdetails;
```

orderid	customerid	itemcode	quantity	dateofpurchase	shippingaddress
39	28	3629	5	2024-02-01	checking today
40	28	391	5	2024-02-01	xyz colony

2 rows in set (0.00 sec)

Amain.py



ADMIN ACCESS

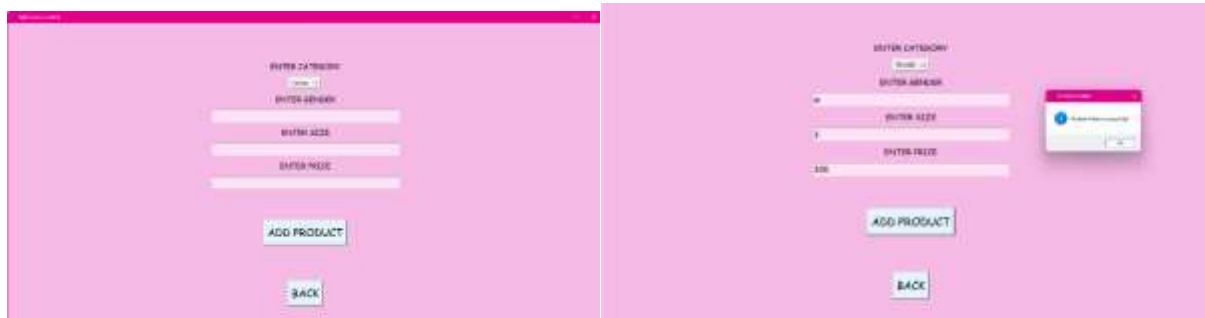
ADD PRODUCT

DELETE PRODUCT

UPDATE STOCK

LOGOUT

Product.py



ENTER CATEGORY

ENTER GENDER

ENTER SIZE

ENTER PRICE

ADD PRODUCT

BACK

ENTER CATEGORY

ENTER GENDER

ENTER SIZE

ENTER PRICE

ADD PRODUCT

BACK

```
mysql> select * from product;
```

typeid	itemcode	category	gender	size	price	autoinc
39	391	household	U	2	200	1
37	373	shoes	U	2	345	3
39	396	household	U	5	350	6
39	397	household	U	5	400	7
36	3617	trouser	U	5	4	17
36	3620	trouser	U	4	500	20
36	3623	trouser	U	3	450	23
36	3625	trouser	U	4	400	25
36	3628	trouser	m	4	340	28
36	3629	trouser	m	4	340	29
35	3530	check	c	4	400	30
36	3632	trouser	m	4	400	32
36	3636	TROUSER	m	3	300	36

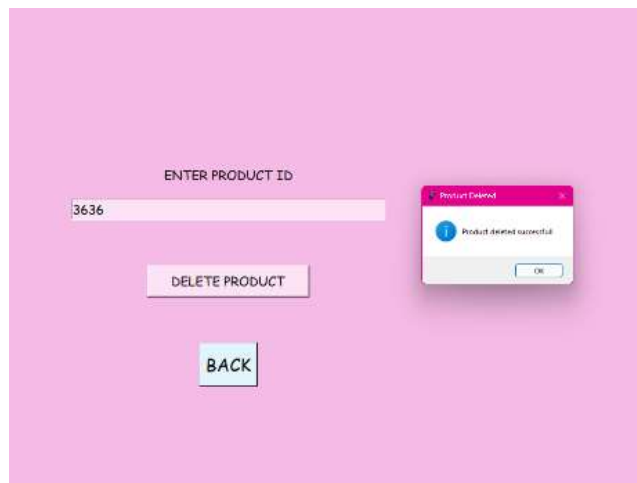
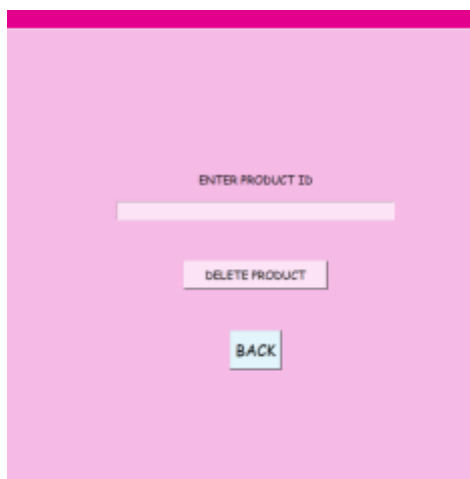
Stock.py

```
mysql> select * from stock;
+-----+-----+
| itemcode | stock |
+-----+-----+
|      391 |    60 |
|     3629 |   475 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> select * from stock;
+-----+-----+
| itemcode | stock |
+-----+-----+
|      391 |    70 |
|     3629 |   475 |
+-----+-----+
2 rows in set (0.00 sec)
```



Delete.py



CONCLUSION

Purpose and Use of the Project

Enhanced Shopping Experience

Advanced Search and Filtering: Users can search by brand, size, color, style, and material with multiple criteria.

Autocomplete Suggestions: Helps users find products quickly.

Personalized User Experience

Machine Learning Recommendations: Suggests products based on past purchases and browsing history.

User Profiles: Saves style preferences and size information.

Informed Purchasing Decisions

Customer Reviews and Ratings: Helps customers make informed decisions and shows the company's responsiveness to feedback.

Efficient Inventory Management

Real-Time Tracking: Ensures accurate product availability.

Data Analytics: Optimizes inventory levels to prevent overstock and stock outs.

Sustainability

Carbon Footprint Calculation: Provides environmental impact information.

Carbon Offset Options: Allows customers to offset their purchases' carbon footprint.

Sustainability Practices: Educates customers about the company's efforts.

Innovative Shopping Tools can be added using GUI

Virtual Fitting Room: Lets customers see how clothes will look on them.

Augmented Reality: Visualizes products in the customer's environment.

Subscription Service: Offers exclusive discounts and early access to new arrivals.

Benefits

Customer Satisfaction: Personalized recommendations and easy search enhance the shopping experience.

Engagement and Loyalty: Social media and subscription services maintain strong customer relationships.

Operational Efficiency: Effective inventory management prevents stock issues.

Sustainability: Eco-friendly practices attract environmentally conscious customers and boost the brand's image.

Threadsorama aims to create a cutting-edge online clothing store that enhances customer experience and promotes sustainability.

BIBLIOGRAPHY

1. Tkinter related information learnt from:

<https://www.geeksforgeeks.org/python-gui-tkinter/>

<https://www.tutorialspoint.com/simple-registration-form-using-python-tkinter>

2. Module definitions: w3schools.com

3. Computer Science for class XII by Sumita Arora